

PROGRESSIVE WEB APPLICATIONS : PWA

Chapitre 4 - Durée 90 minutes



OBJECTIFS

- ✓ Concevoir une « Progressive Web Application »
- ✓ Utiliser les fonctionnalités natives

AGENDA

- ✓ Aperçu d'une « Progressive Web Application »
- ✓ Accès aux fonctionnalités natives
- ✓ Fonctionnalités Avancées

UN PEU D'HISTOIRE



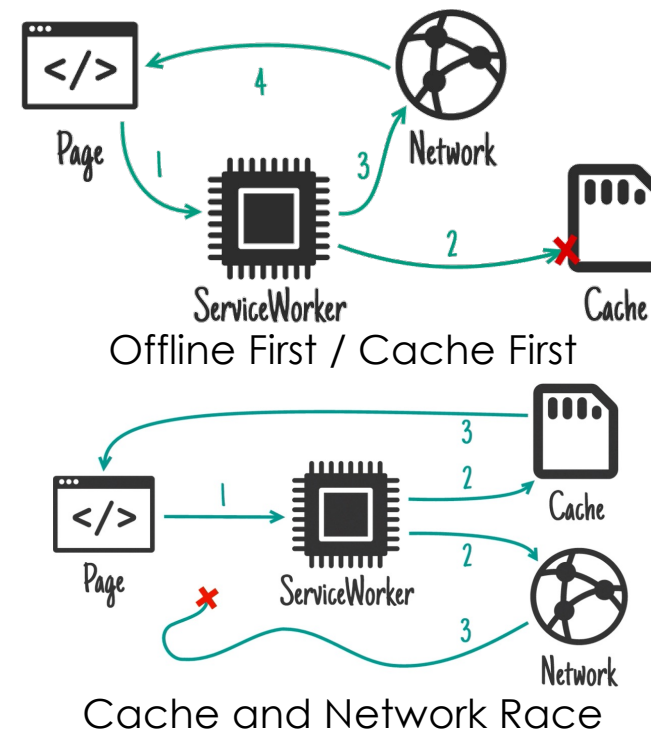
ANATOMIE D'UNE PWA (1)



ANATOMIE D'UNE PWA (2)



© Mohamed-Bécha Kaâniche 2015-2023



ANATOMIE D'UNE PWA (3)

```
<!doctype html>
<html lang="en">
  <head>
    <meta charset="utf-8" />
    <title>Simple PWA</title>
    <link rel="canonical" href="https://www.yourappdomain.tld" />
    <meta name="viewport" content="width=device-width, initial-scale=1" />
    <link rel="manifest" href="manifest.json" />
    <link rel="stylesheet" href="styles/main.css" />
    <script type="module" defer="defer" src="scripts/main.js"></script>
    ...
  </head>
  <body>
    ...
  </body>
</html>
```

ANATOMIE D'UNE PWA (4)

✓ Autres balises importantes descendantes de la balise « head » :

```
<link rel="apple-touch-icon" sizes="180x180" href="./apple-touch-icon.png" />
<link rel="shortcut icon" href="./favicon.ico" /> <!--48x48-->
<link rel="icon" type="image/png" sizes="32x32" href="./favicon-32x32.png" />
<link rel="icon" type="image/png" sizes="16x16" href="./favicon-16x16.png" />
<meta name="theme-color" content="#000000" />
<link rel="mask-icon" href="./safari-pinned-tab.svg" color="#000000" />
<meta name="description" content="Simple Progressive Web App (PWA) template" />
<meta name="robots" content="noodp" />
<meta name="author" content="Mohamed-Bécha Kaâniche">
<meta name="keywords" content="PWA, SPA, ..." />
```

ANATOMIE D'UNE PWA (5)

✓ Le fichier « manifest.json » :

```
{  
  "name": "Your long PWA Name",  
  "short_name": "Short Name",  
  "display": "standalone",  
  "start_url": "/",  
  "scope": ".",  
  "theme_color": "#000",  
  "background_color": "#000",  
  "icons": [  
    {  
      "src": "images/icons/icon-36x36.png",  
      "sizes": "36x36",  
      "type": "image/png"  
    }, ...  
  ]  
}
```

ANATOMIE D'UNE PWA (6)

✓ Dans le fichier « scripts/main.js », appeler le « service worker » :

```
if ("serviceWorker" in navigator) {  
  window.addEventListener("load", () => {  
    navigator.serviceWorker &&  
    navigator.serviceWorker.register("./sw.js");  
  });  
}
```

ANATOMIE D'UNE PWA (7)

✓ Le fichier « sw.js » :

```
const cacheName = "cache1"; // Change value to force update
self.addEventListener("install", event => {
  // Kick out the old service worker
  self.skipWaiting();
  event.waitUntil(
    caches.open(cacheName).then(cache => {
      return cache.addAll([
        "/",
        "index.html",
        "images/logo.png",
        "scripts/main.js",
        "styles/main.css", ...
      ]);
    })
  );
});
```

ANATOMIE D'UNE PWA (8)

✓ Le fichier « sw.js » (Suite) :

```
self.addEventListener("activate", event => {  
  // Delete any non-current cache  
  event.waitUntil(  
    caches.keys().then(keys => {  
      Promise.all(  
        keys.map(key => {  
          if (![cacheName].includes(key)) {  
            return caches.delete(key);  
          }  
        })  
      })  
    })  
  );  
});
```

ANATOMIE D'UNE PWA (9)

✓ Le fichier « sw.js » (Suite) (Offline First / Cache First Strategy) :

```
self.addEventListener("fetch", event => {  
  event.respondWith(  
    caches.open(cacheName).then(cache => {  
      return cache.match(event.request).then(response => {  
        return response || fetch(event.request).then(networkResponse => {  
          cache.put(event.request, networkResponse.clone());  
          return networkResponse;  
        });  
      });  
    })  
  })  
});
```

ACCÈS AUX FONCTIONNALITÉS NATIVES (1)

- ✓ Accès aux caméras, aux microphones et aux flux de données (WebRTC) :
 - ✓ Pour connaître les contraintes multimédia supportés par le périphérique courant:
`navigator.mediaDevices.getSupportedConstraints()`
 - ✓ Pour obtenir un « `MediaStream` » depuis un ou plusieurs capteurs multimédia :
`navigator.mediaDevices.getUserMedia(constraints)`
 - ✓ Pour ouvrir un flux de données « `RTCDataChannel` » :

```
const pc = new RTCPeerConnection();  
const dc = pc.createDataChannel("my channel");  
dc.onmessage = (event) => { console.log(`received: ${event.data}`); };  
dc.onopen = () => { console.log("datachannel open"); };  
dc.onclose = () => { console.log("datachannel close"); };
```

ACCÈS AUX FONCTIONNALITÉS NATIVES (2)

✓ Exemples d'obtention de flux multimédia:

```
const stream= await navigator.mediaDevices.getUserMedia({audio: false, video: true})
```

//contrainte pour caméra arrière

```
{ video: { ..., facingMode: { exact: 'environment' } } }
```

//contrainte pour caméra avant

```
{ video : { ..., facingMode: 'user' } }
```

//contraintes avancées

```
{ video: { minAspectRatio: 1.333, minFrameRate: 30, width: 1280, height: 720 } }
```

ACCÈS AUX FONCTIONNALITÉS NATIVES (3)

✓ HTML5:

```
<button id="get-access">Get access to camera</button>  
<video autoplay></video>
```

✓ JavaScript:

```
document.querySelector('#get-access').addEventListener('click', async function init(e) {  
  try {  
    const stream = await navigator.mediaDevices.getUserMedia({ audio: false, video: true})  
    const videoTracks = stream.getVideoTracks()  
    const track = videoTracks[0] alert(`Getting video from: ${track.label}`)  
    document.querySelector('video').srcObject = stream  
    document.querySelector('#get-access').setAttribute('hidden', true)  
    setTimeout(() => { track.stop() }, 3 * 1000)  
  } catch (error) {  
    alert(`${error.name}`)  
    console.error(error)  
  }  
});
```

ACCÈS AUX FONCTIONNALITÉS NATIVES (4)

❖ Détection de l'événement « foreground » :

```
var target = document.getElementById('target');
function handleVisibilityChange() {
    var timeBadge = new Date().toString().split(' ')[0];
    var newState = document.createElement('p');
    newState.innerHTML = '' + timeBadge + 'Visibility changed to ' + (document["hidden"] ? 'hidden' : 'visible') + '.';
    target.appendChild(newState);
}
document.addEventListener("visibilityChange", handleVisibilityChange, false);
if (hidden in document) {
    document.getElementById('status').innerHTML = document["hidden"] ? 'hidden' : 'visible';
}
```

ACCÈS AUX FONCTIONNALITÉS NATIVES (5)

```
document.addEventListener('visibilitychange', () => {  
  if (document.visibilityState === 'visible') {  
    // Your code here...  
  }  
});  
  
const getState = () => {  
  if (document.visibilityState === 'hidden') {  
    return 'hidden';  
  }  
  if (document.hasFocus()) {  
    return 'active';  
  }  
  return 'passive';  
};
```

ACCÈS AUX FONCTIONNALITÉS NATIVES (6)

```
let state = getState();
const logStateChange = (nextState) => {
  const prevState = state;
  if (nextState !== prevState) {
    console.log(`State change: ${prevState} >>> ${nextState}`);
    state = nextState;
  }
};
const opts = {capture: true};
['pageshow', 'focus', 'blur', 'visibilitychange', 'resume'].forEach((type) => {
  window.addEventListener(type, () => logStateChange(getState(), opts);
});
window.addEventListener('freeze', () => { logStateChange('frozen'); }, opts);
window.addEventListener('pagehide', (evt) => {
  logStateChange(evt.persisted ? 'frozen' : 'terminated');
}, opts);
```

FONCTIONNALITÉS AVANCÉES (1)

❖ Notifications Push : Push API & Notification API :

❖ Dans le fichier « manifest.json », ajouter dans la clé permissions:

```
"permissions": [ "notifications" ]
```

❖ Réagissez à l'événement déclenchant l'abonnement :

```
if (!("Notification" in window)) {  
    alert("notifications not supported");  
} else if (Notification.permission === "granted") {  
    const notification = new Notification("welcome back!");  
} else if (Notification.permission !== "denied") {  
    Notification.requestPermission().then((result) => {  
        if (result === "granted") {  
            new Notification('welcome title', { body:'...', icon: '...' });  
        }  
    });  
}
```

FONCTIONNALITÉS AVANCÉES (2)

❖ Dans le fichier « scripts/main.js », ajoutez après l'enregistrement du « service worker » :

```
navigator.serviceWorker.ready
  .then((registration) => {
    return registration.pushManager.getSubscription().then(async function(subscription) {
      if (subscription) { // If a subscription was found, return it.
        return subscription;
      }
      const response = await fetch('./pathToRetrieveYouServerPublicKey');
      const serverPushAPIPublicKey = await response.text();
      const convertedPublicKey = urlBase64ToUint8Array(serverPushAPIPublicKey);
      return registration.pushManager.subscribe({
        userVisibleOnly: true,
        applicationServerKey: convertedPublicKey
      });
    });
  }).then((subscription) => { // Send the subscription details to the server using the Fetch API.
    fetch('./register', { method: 'post',
      headers: { 'Content-type': 'application/json' },
      body: JSON.stringify({ subscription: subscription })
    });
  });
```

FONCTIONNALITÉS AVANCÉES (3)

❖ Dans le fichier « sw.js » du service worker, réagissez à l'événement « push » :

```
self.addEventListener('push', (evt) => {  
    const notif = evt.data ? evt.data.json() : {title:'...',body:'...'};  
    event.waitUntil(  
        self.registration.showNotification(notif.title, {body: notif.body})  
    );  
});
```

FONCTIONNALITÉS AVANCÉES (4)

❖ Gestion des badges:

```
self.addEventListener('push', (evt) => {  
  const messages = evt.data.json();  
  const unreadCount = messages.unreadCount;  
  if (navigator.setAppBadge) {  
    if (unreadCount && unreadCount > 0){  
      navigator.setAppBadge(unreadCount);  
    } else {  
      navigator.clearAppBadge();  
    }  
  }  
});
```

LES SERVICES DE GÉOLOCALISATION

Chapitre 5 - Durée 90 minutes



OBJECTIFS

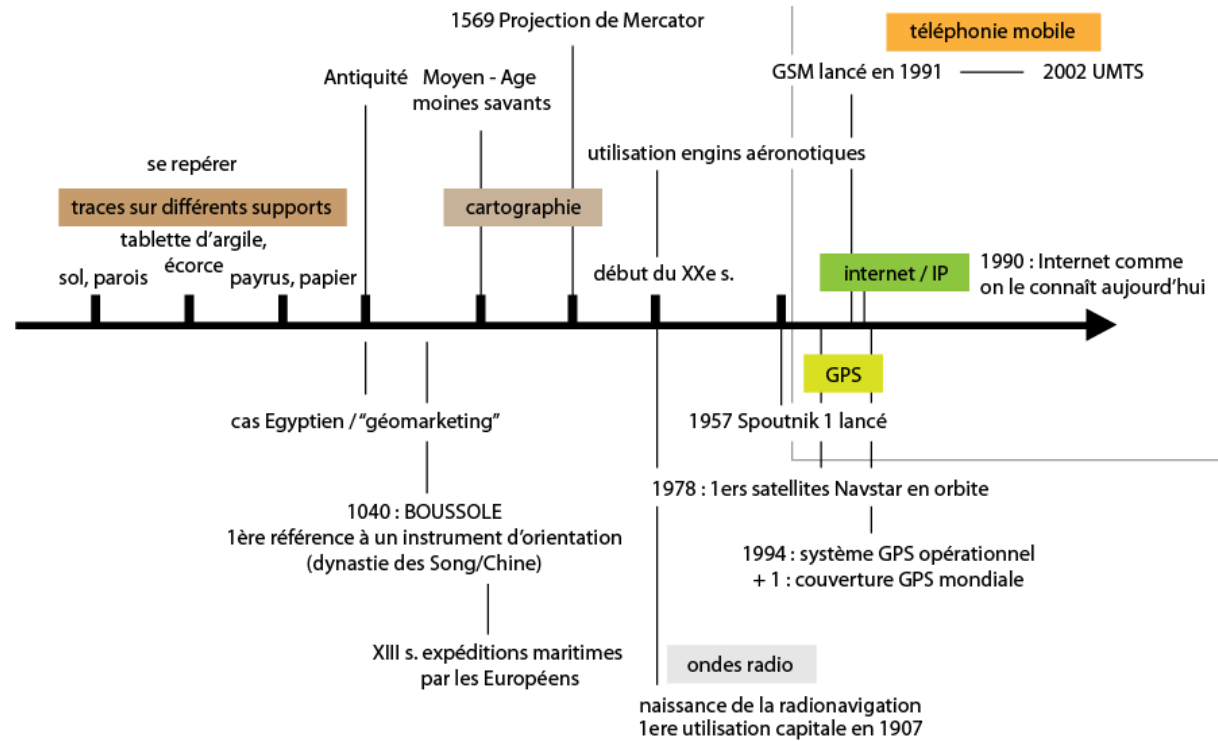
- ✓ Décrire les services de géolocalisation (Location Based Services)
- ✓ Utiliser les services de géolocalisation dans une PWA

AGENDA

- ✓ Services de géolocalisation
- ✓ Services de géolocalisation pour une PWA

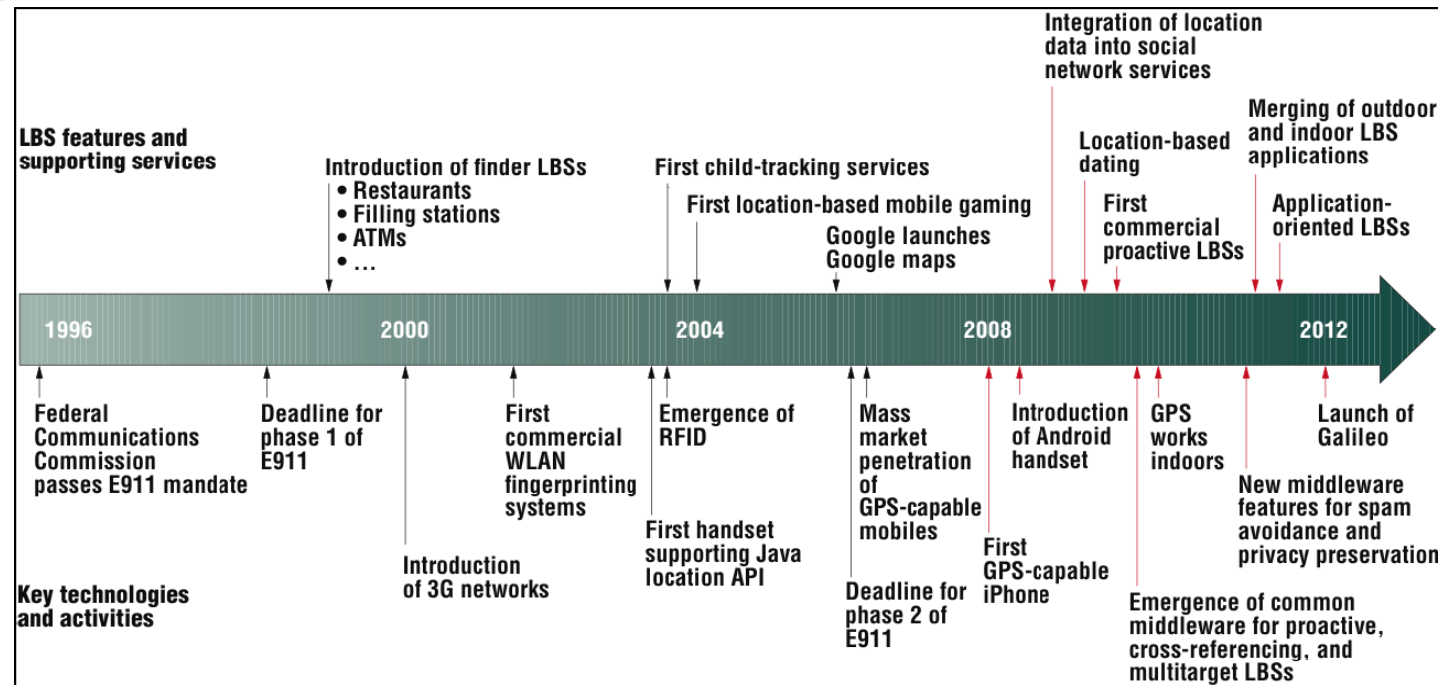
SERVICES DE GÉOLOCALISATION

- ✓ Procédé permettant de positionner un objet (une personne, une information...) sur un plan ou une carte à l'aide de ses coordonnées géographiques.
 - ✓ En marketing : localisation du destinataire d'un message marketing par son téléphone portable ou via un site Web.
- ✓ Intérêts :
 - ✓ adapter la communication et l'offre
 - ✓ cibler un consommateur de plus en plus nomade (récupération de données utiles, lieu...)
 - ✓ Offrir une expérience utilisateur basée sur la géolocalisation (p.ex. la réalité augmentée)



SERVICES DE GÉOLOCALISATION

Un peu d'histoire



SERVICES DE GÉOLOCALISATION

Un peu d'histoire

SERVICES DE GÉOLOCALISATION

- ✓ Domaines d'applications
 - ✓ Militaires (missiles, stratégie de terrain, déplacement de troupes...)
 - ✓ Transports (navigation)
 - ✓ Traçabilité (marchandises...)
 - ✓ Urgences, Hôpitaux (repérage de blessés...)
 - ✓ Sécurité (vols en magasins, vols de voitures...)
 - ✓ Guide (musées, bibliothèques...)
 - ✓ Géomarketing

SERVICES DE GÉOLOCALISATION

✓ Technologies utilisées

- ✓ GPS

- ✓ Téléphonie : GSM, UMTS, LTE(-B)

- ✓ Internet: UDP/TCP/IP

- ✓ Wifi, Bluetooth

- ✓ RFID, NFC, iBeacons (Bluetooth Low Energy)

➔ Différentes zones et modes d'utilisation.

PWA – SERVICES DE GÉOLOCALISATION

✓ Dans le fichier « manifest.json », ajouter dans la clé permissions:

```
"permissions": [ "geolocation" ]
```

✓ Initialisation:

```
if ( navigator.permissions && navigator.permissions.query) { //try
permissions APIs first
  navigator.permissions.query({name:'geolocation'}).then(function(result){
    // Will return ['granted', 'prompt', 'denied']
    const permission = result.state;
    if ( permission === 'granted' || permission === 'prompt' ) {...}
  });
} else if (navigator.geolocation) { //then Navigation APIs
  ...
}
```

PWA – SERVICES DE GÉOLOCALISATION

✓ Méthode : `navigator.geolocation.getCurrentPosition`

```
navigator.geolocation.getCurrentPosition(geolocationSuccess,  
                                         [geolocationError],  
                                         [geolocationOptions]);
```

- ✓ `geolocationSuccess`: La fonction callback au quelle est passée la position courante.
- ✓ `geolocationError`: (Optionnel) La fonction callback exécuté en cas d'erreur.
- ✓ `geolocationOptions`: (Optionnel) les options de géolocalisation.

PWA – SERVICES DE GÉOLOCALISATION

```
var onSuccess = function(position) {  
    alert( 'Latitude: '      + position.coords.latitude      + '\n' +  
          'Longitude: '     + position.coords.longitude     + '\n' +  
          'Altitude: '      + position.coords.altitude      + '\n' +  
          'Accuracy: '      + position.coords.accuracy       + '\n' +  
          'Altitude Accuracy: ' + position.coords.altitudeAccuracy + '\n' +  
          'Heading: '       + position.coords.heading        + '\n' +  
          'Speed: '         + position.coords.speed           + '\n' +  
          'Timestamp: '     + position.timestamp              + '\n');  
};  
  
function onError(error) {  
    alert('code: ' + error.code + '\n' + 'message: ' + error.message + '\n');  
}  
  
navigator.geolocation.getCurrentPosition(onSuccess, onError);
```

PWA – SERVICES DE GÉOLOCALISATION

✓ Méthode `navigator.geolocation.watchPosition`

```
var watchId =  
    navigator.geolocation.watchPosition (geolocationSuccess,  
                                          [geolocationError],  
                                          [geolocationOptions]);
```

- ✓ `geolocationSuccess`: La fonction callback au quelle est passée la position courante.
- ✓ `geolocationError`: (Optionnel) La fonction callback exécuté en cas d'erreur.
- ✓ `geolocationOptions`: (Optionnel) les options de géolocalisation.
- ✓ Retourne une chaine de caractères identifiant la surveillance de position

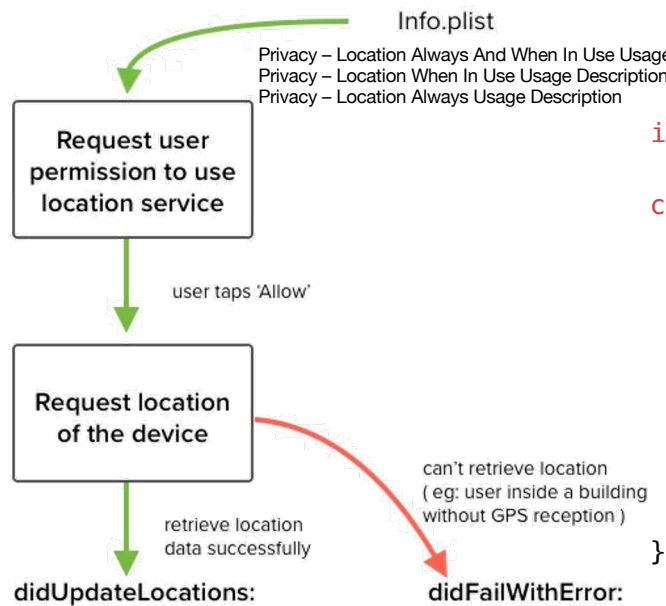
PWA – SERVICES DE GÉOLOCALISATION

```
function onSuccess(position) {  
    var element = document.getElementById('geolocation');  
    element.innerHTML = 'Latitude: ' + position.coords.latitude + '<br />' +  
        'Longitude: ' + position.coords.longitude + '<br />' +  
        '<hr />' + element.innerHTML;  
}  
  
function onError(error) {  
    alert('code: ' + error.code + '\n' + 'message: ' + error.message + '\n');  
}  
  
// options: throw an error if no update is received every 30 seconds.  
var watchID = navigator.geolocation.watchPosition(onSuccess, onError, {timeout: 30000});
```

PWA – SERVICES DE GÉOLOCALISATION

- ✓ Options de géolocalisation :
 - { maximumAge: 3000, timeout: 5000, enableHighAccuracy: true };
- ✓ Méthode navigator.geolocation.clearWatch(watchID);
- ✓ Position
 - ✓ coords
 - ✓ latitude: Latitude in decimal degrees. (Number)
 - ✓ longitude: Longitude in decimal degrees. (Number)
 - ✓ altitude: Height of the position in meters above the ellipsoid. (Number)
 - ✓ accuracy: Accuracy level of the latitude and longitude coordinates in degrees. (Number)
 - ✓ altitudeAccuracy: Accuracy level of the altitude coordinate in meters. (Number)
 - ✓ heading: Direction of travel, specified in degrees counting clockwise relative to the true north. (Number)
 - ✓ speed: Current ground speed of the device, specified in meters per second. (Number)
 - ✓ timestamp

iOS – SERVICES DE GÉOLOCALISATION



```
import CoreLocation
```

```
class LocationDataManager : NSObject, ObservableObject, CLLocationManagerDelegate {
    var locationManager = CLLocationManager()
    @Published var authorizationStatus: CLAuthorizationStatus?

    override init() {
        super.init()
        locationManager.delegate = self
    }
}
```

```
//... (cf. 2 prochaines diapositives)
```

iOS – SERVICES DE GÉOLOCALISATION

```
func locationManagerDidChangeAuthorization(_ manager: CLLocationManager) {  
    switch authorizationStatus {  
    case .authorizedWhenInUse: // Location services are available.  
        // Insert code here of what should happen when Location services are authorized  
        authorizationStatus = .authorizedWhenInUse  
        locationManager.requestLocation() //or handle .startUpdatingLocation()/.stopUpdatingLocation() flow  
        break  
    case .restricted: // Location services currently unavailable.  
        // Insert code here of what should happen when Location services are NOT authorized  
        authorizationStatus = .restricted  
        break  
    case .denied: // Location services currently unavailable.  
        // Insert code here of what should happen when Location services are NOT authorized  
        authorizationStatus = .denied  
        break  
    case .notDetermined: // Authorization not determined yet.  
        authorizationStatus = .notDetermined  
        manager.requestWhenInUseAuthorization()  
        break  
    default  
        break  
    }  
}
```

ios – SERVICES DE GÉOLOCALISATION

```
func locationManager(_ manager: CLLocationManager, didUpdateLocations locations: [CLLocation]) {  
    // Insert code to handle location updates  
    if let location = locations.last {  
        let latitude = location.coordinate.latitude  
        let longitude = location.coordinate.longitude  
        let degreeAccuracy = location.horizontalAccuracy  
        let altitude = location.altitude  
        let altitudeAccuracy = location.verticalAccuracy  
        let speed = location.speed  
        let speedAccuracy = location.speedAccuracy  
        let course = location.course //in degree counting clockwise relative to true north  
        let courseAccuracy = location.courseAccuracy  
        let floor = location.floor  
        let timeSinceEpoch = location.timestamp.timeIntervalSince1970  
        let distance = location.distance(from: locations.first!) //lateral distance  
    }  
}  
  
func locationManager(_ manager: CLLocationManager, didFailWithError error: Error) {  
    print("error: \(error.localizedDescription)")  
}
```

iOS – SERVICES DE GÉOLOCALISATION

```
struct YourLocationView: View {
    @StateObject var locationManager = LocationDataManager()
    var body: some View {
        VStack {
            switch locationManager.locationManager.authorizationStatus {
            case .authorizedWhenInUse: // Location services are available.
                // Insert code here of what should happen when Location services are authorized
                Text("Your current location is:")
                Text("Latitude: \(locationDataManager.locationManager.location?.coordinate.latitude.description ?? "Error loading")")
                Text("Longitude: \(locationDataManager.locationManager.location?.coordinate.longitude.description ?? "Error loading")")

            case .restricted, .denied: // Location services currently unavailable.
                // Insert code here of what should happen when Location services are NOT authorized
                Text("Current location data was restricted or denied.")
            case .notDetermined: // Authorization not determined yet.
                Text("Finding your location...")
                ProgressView()
            default:
                ProgressView()
            }
        }
    }
}
```